

PORTADA

Arquitectura de Software

Semana 13 — Implementación de Capas y Mapeo Objeto-Relacional (ORM)

De las capas en el papel al código que las implementa — cierre de la Unidad 3 · Universidad de Antioquia · Ingeniería de Sistemas ·
2554608 · 2026-2



APERTURA

9 servidores web.
1.300 millones de páginas vistas al mes.

Stack Overflow, 2016 — y ni un solo ORM completo en su capa de persistencia.



TRANSICIÓN

Esa cifra no cuadra con la intuición

La reacción esperada frente a "1.300 millones de páginas vistas al mes" es pensar en cientos de servidores en la nube, balanceadores de carga elaborados y una factura de infraestructura enorme. Stack Overflow, en cambio, documentó públicamente que servía ese tráfico con un puñado de máquinas.

Parte de esa eficiencia no vino de tener más hardware, sino de una decisión arquitectónica muy concreta sobre cómo se implementa la capa de persistencia — la capa que hoy es el centro de la sesión.

Stack Overflow / Dapper, 2016

Empresa	Stack Exchange Inc. — red de sitios que incluye Stack Overflow
Documentación	Serie de artículos <i>"What it takes to run Stack Overflow"</i> , del ingeniero Nick Craver , blog de ingeniería de la compañía, 2016
Tráfico reportado	Del orden de 1.300 millones de páginas vistas al mes en toda la red de sitios
Infraestructura	Del orden de 9 servidores web para toda la red — mucho menor de lo que la intuición esperaría
Capa de presentación	ASP.NET MVC
Capa de persistencia	Dapper , micro-ORM propio, en vez de un ORM completo como Entity Framework

Uno de los pocos casos de arquitectura de producción documentado públicamente con este nivel de detalle técnico por el propio equipo que la construyó.

CASO · LAS CAPAS, EN CÓDIGO REAL

Presentación, negocio y persistencia — separadas de verdad

La arquitectura que documentó Craver mantiene una separación de capas clásica, la misma que ya conocen desde el patrón **Layers** (Semana 7), pero llevada a decisiones concretas de implementación:

- **Capa de presentación:** ASP.NET MVC — vistas, controladores de UI, serialización de las respuestas HTTP.
- **Capa delgada de lógica de negocio/servicios:** reglas propias del dominio (reputación, votos, badges, moderación), independiente de cómo se guardan los datos.
- **Capa de acceso a datos vía Dapper:** mapea directamente los resultados de SQL a las clases del modelo de dominio, sin capas intermedias de abstracción pesadas.

La separación no es solo prolija en el diagrama: cada capa vive en su propio conjunto de clases/proyectos dentro del código.

CASO · LA DECISIÓN DETRÁS DE DAPPER

Escribir más SQL a cambio de más control

En vez de adoptar un ORM completo como **Entity Framework** o **Hibernate**, el equipo de Stack Overflow — Sam Saffron y Marc Gravell — construyó **Dapper**, un "micro-ORM" open source que solo hace una cosa: mapear el resultado de una consulta SQL a objetos **POCO** (Plain Old CLR Objects) del modelo de dominio.

Fue explícitamente una decisión arquitectónica de rendimiento y control, no un accidente ni una limitación técnica: aceptar escribir el SQL a mano (menos "productividad" de alto nivel) para evitar los problemas de rendimiento típicos de los ORM completos.

La elección de "cuánto ORM" usar en la capa de persistencia es, en sí misma, una decisión de arquitectura — no un simple detalle de implementación que se resuelve solo.

TRANSICIÓN

De la anécdota a la disciplina completa

Para entender por qué esa decisión sobre Dapper importa, hay que aterrizar dos cosas que hasta ahora han visto sobre todo en teoría: qué es exactamente cada capa de una aplicación implementada, y qué hace un ORM técnicamente cuando media entre esas capas y la base de datos.

Esta es la última sesión de la Unidad 3 — Desarrollo de Software basado en Arquitectura. Cerramos viendo cómo el patrón Layers, que ya conocen desde la Semana 7, se convierte en carpetas, proyectos y clases reales.

CONCEPTO · REPASO

Layers (Semana 7): de patrón a implementación

En la Semana 7 vieron **Layers** como patrón POSA: organizar el sistema en niveles horizontales, donde cada capa solo puede depender de la capa inmediatamente inferior, y cada una tiene una responsabilidad exclusiva.

Hoy bajamos ese patrón al terreno concreto: ¿cómo se ve "una capa" cuando dejas el diagrama y abres el proyecto en el editor de código? Normalmente como una carpeta o un proyecto separado (ej. `Presentation/` , `Services/` , `Data/`), con clases que solo llaman a las de la capa inferior — nunca al revés, y nunca saltándose una capa intermedia.



CONCEPTO

Las cinco capas de una aplicación empresarial típica

Capa	Responsabilidad
Presentación	Interfaz con el usuario: vistas, formularios, serialización de las respuestas (HTML, JSON).
Controlador	Recibe las solicitudes y orquesta la interacción entre presentación y lógica de negocio.
Lógica de negocio	Reglas y procesos propios del dominio del problema, independientes de cómo se muestran o se guardan los datos.
Modelo de dominio	Las entidades y objetos que representan los conceptos del negocio.
Persistencia	Guarda y recupera datos de un almacenamiento durable (una base de datos relacional, en esta sesión).

En el caso de Stack Overflow (s05), estas cinco capas se agrupan en tres bloques de código: presentación+controlador (ASP.NET MVC), negocio (servicios delgados) y persistencia (Dapper + modelo de dominio).

CONCEPTO · PRECISIÓN NECESARIA

Controlador: el intermediario, no el que decide

La **capa de Controlador** merece una aclaración porque ya la vieron en otro contexto: en el patrón **MVC** (Semana 6), el Controlador es exactamente el intermediario entre el Modelo y la Vista — recibe la solicitud del usuario, decide a qué componente de negocio delegar, y prepara la respuesta para la Vista.

Un error común de implementación es que el Controlador termine haciendo el trabajo de la capa de negocio (validaciones complejas, cálculos del dominio) en vez de solo orquestar — eso rompe la separación de responsabilidades y convierte al controlador en un "God Object" (Semana 11) de facto.

CONCEPTO · PRECISIÓN NECESARIA

Modelo de dominio $\neq$ tablas de la base de datos

Es tentador asumir que el **modelo de dominio** (las clases que representan Usuario, Pregunta, Respuesta, Voto) es simplemente "un espejo" de las tablas de la base de datos. No siempre es así:

- Una clase del dominio puede combinar datos de varias tablas relacionadas.
- Una tabla puede dividirse en varias clases del dominio, o viceversa.
- El modelo de dominio se diseña para expresar el negocio con claridad; el esquema relacional se diseña para almacenar datos de forma eficiente y consistente — son preocupaciones distintas que no siempre coinciden en forma.

Esa diferencia de forma entre el mundo de objetos y el mundo relacional es precisamente el problema que resuelve (o intenta resolver) un ORM.

CONCEPTO

¿Qué es el Mapeo Objeto-Relacional (ORM)?

El Mapeo Objeto-Relacional (Object-Relational Mapping, ORM) es la técnica y la herramienta que traducen entre el **modelo de objetos de la aplicación (clases, atributos, relaciones)** y el **modelo relacional de la base de datos (tablas, filas, claves foráneas)**.

El problema que resuelve tiene nombre propio: el **"impedance mismatch" objeto-relacional** — el desajuste estructural entre cómo un lenguaje orientado a objetos representa la información (objetos con identidad, herencia, referencias) y cómo un motor relacional la representa (filas planas, claves primarias/foráneas, sin herencia nativa).



CONCEPTO

Qué automatiza un ORM, en concreto

Cuando se habla de "usar un ORM" en la capa de persistencia, normalmente se refiere a que la herramienta se encarga de tres tareas que, sin ella, el desarrollador tendría que escribir a mano:

- **Generación de SQL:** traduce operaciones sobre objetos (crear, leer, actualizar, borrar) en las sentencias SQL correspondientes.
- **Tracking de cambios (change tracking):** detecta qué propiedades de un objeto cambiaron desde que se cargó, para generar solo el UPDATE necesario.
- **Lazy loading:** carga automáticamente datos relacionados (ej. las respuestas de una pregunta) solo cuando el código realmente los usa, no de entrada.

Estas tres tareas son las que un ORM completo automatiza — y las que un micro-ORM, como verán a continuación, decide no automatizar.

CONTRAEJEMPLO · DE VUELTA AL CASO

ORM completo vs. micro-ORM: Entity Framework vs. Dapper

	ORM completo (Entity Framework, Hibernate)	Micro-ORM (Dapper)
Genera SQL automáticamente	Sí	No — el desarrollador escribe el SQL
Tracking de cambios	Sí, automático	No existe
Lazy loading	Sí, automático	No existe
Qué automatiza	Todo el ciclo objeto ↔ SQL	Solo el mapeo de resultados a objetos POJO
Control sobre el SQL ejecutado	Indirecto, generado por la herramienta	Total — el desarrollador escribe cada consulta

Stack Overflow eligió deliberadamente la columna derecha: menos automatización, más control sobre lo que realmente se ejecuta contra la base de datos.

CONCEPTO · EL TRADE-OFF EXPLÍCITO

Productividad vs. rendimiento: no hay opción gratis

La elección entre ORM completo y micro-ORM no tiene una respuesta universalmente correcta — es un trade-off arquitectónico real:

- **A favor del ORM completo:** mayor productividad inicial, menos código repetitivo, curva de aprendizaje más simple para el equipo.
- **A favor del micro-ORM:** control total sobre el SQL ejecutado, sin overhead de generar SQL dinámicamente ni de rastrear el estado de cada objeto en memoria — mejor rendimiento en sistemas de tráfico muy alto.

Stack Overflow, con 1.300 millones de páginas vistas al mes sobre solo 9 servidores web, es evidencia de que esa apuesta por el control y el rendimiento, sostenida en el tiempo, puede ser la decisión correcta a esa escala.

CONCEPTO

El problema N+1: cuando el ORM se vuelve el problema

El **problema de consultas N+1** es el ejemplo más común y más citado de cómo un ORM completo, mal usado, puede degradar el rendimiento de forma severa — precisamente en la capa de persistencia.

Ocurre cuando el código pide una lista de N elementos (ej. "todas las preguntas de un usuario") y, por cada uno de esos N elementos, el ORM dispara automáticamente **una consulta SQL adicional** para cargar un dato relacionado (ej. las respuestas de cada pregunta) mediante lazy loading — en vez de traer todo en una sola consulta bien construida.

El resultado: una operación que debería costar una sola consulta SQL termina costando N+1 consultas — cientos o miles, si N es grande.

CONCEPTO · POR QUÉ IMPORTA AQUÍ

N+1 explica, en parte, la decisión de Dapper

El lazy loading automático de un ORM completo es cómodo de escribir, pero es exactamente el mecanismo que abre la puerta al problema N+1: el desarrollador escribe una línea de código simple (iterar sobre una colección relacionada) sin ver, en ese momento, cuántas consultas SQL reales se están disparando por detrás.

Con un micro-ORM como Dapper, ese riesgo desaparece por construcción: como no hay lazy loading automático, el desarrollador escribe explícitamente el JOIN o la consulta que necesita, y ve exactamente cuántas idas a la base de datos está generando su código.

No es que Dapper "resuelva" el problema N+1 con magia — es que el diseño de un micro-ORM hace visible el costo real de cada acceso a datos, en vez de ocultarlo detrás de una API cómoda.

INTERACCIÓN

¿Cómo están implementando ustedes sus capas?

En equipos, revisen su proyecto de CodeF@ctory y respondan:

1. ¿Pueden señalar, en su código, dónde termina la capa de presentación/controlador y dónde empieza la lógica de negocio?
¿Está realmente separado, o mezclado en los mismos archivos?
2. ¿Cómo acceden a su base de datos hoy: con un ORM completo (Entity Framework, Hibernate, u otro), un micro-ORM, o SQL directo sin ninguna herramienta?
3. Con lo visto hoy, ¿identifican algún riesgo de problema N+1 en alguna parte de su acceso a datos actual?

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. El patrón **Layers** (Semana 7) se implementa en código como carpetas/proyectos separados: Presentación, Controlador, Lógica de Negocio, Modelo de Dominio y Persistencia, cada uno con responsabilidad exclusiva.
2. El **modelo de dominio no es igual a las tablas** de la base de datos — esa diferencia de forma es el "impedance mismatch" objeto-relacional que un **ORM** busca resolver.
3. Un ORM completo (Entity Framework, Hibernate) automatiza generación de SQL, tracking de cambios y lazy loading; un **micro-ORM** como **Dapper** solo mapea resultados a objetos POCO, dejando el SQL explícito.
4. Stack Overflow (2016, ~1.300 millones de páginas vistas al mes con ~9 servidores web) eligió Dapper deliberadamente: más control y rendimiento, a cambio de menos automatización.
5. El **problema N+1** es el riesgo típico del lazy loading automático de un ORM completo: una operación que debería ser una consulta se convierte en N+1 consultas SQL.

SÍNTESIS · CIERRE DE LA UNIDAD 3

Cuatro sesiones, una sola unidad: Desarrollo de SW basado en Arquitectura

Con esta sesión cerramos la Unidad 3 completa, cuatro sesiones desde la Semana 12 hasta hoy:

- **Desarrollo basado en Componentes** (Semana 12) — construir sistemas ensamblando piezas de software reutilizables e independientes, en vez de escribir todo desde cero.
- **Proceso de Desarrollo orientado a la Arquitectura** (Semana 12) — cómo la arquitectura guía y se integra al ciclo de vida del desarrollo, no como un documento aparte sino como una actividad continua del proceso.
- **Middleware y Frameworks** (Semana 13) — el software intermedio que conecta componentes y sistemas distintos, y los frameworks que dan estructura y andamiaje al código de la aplicación.
- **Implementación de Capas y ORM** (hoy) — cómo el patrón Layers se aterriza en código real, y cómo la elección de "cuánto ORM" usar en la persistencia es en sí misma una decisión arquitectónica, con el caso de Stack Overflow y Dapper.

Toda la Unidad 3 respondió una sola pregunta: una vez que ya se decidió la arquitectura (Unidades 1 y 2), ¿cómo se construye realmente el software que la implementa? Componentes, proceso, middleware, frameworks y capas son las piezas concretas de esa respuesta.

CIERRE

Para la próxima semana

- Repasar las cinco capas vistas hoy (presentación, controlador, lógica de negocio, modelo de dominio, persistencia) y ubicarlas explícitamente en el código de su proyecto de CodeF@ctory.
- Revisar con su equipo qué herramienta de acceso a datos están usando hoy, y si conviene ajustarla a la luz del trade-off ORM completo vs. micro-ORM visto en esta sesión.

La Semana 14 abre la Unidad 4 — Los Servicios Web en la Arquitectura — con Esquemas para el uso de servicios web y Arquitecturas Orientadas a Servicios (SOA).

